



Week 3 Worksheet: Behaviour Trees

Overview

The lab Objectives are:

- Implement a basic Behaviour Tree using simple control nodes
- Expand the behaviour tree using additional control nodes
- Understand the value of Conditions in terms of AI reactivity

Getting Started

From the Week 3 content on MLS, download '**Behaviour Trees - C++**' from the "Week 3 Lab Resources" section. Extract the contents of the zip file. Open the Visual Studio Project.

Note: Visual Studio is the recommended software to use, although any C++ development environment should be sufficient.

Run the project.

You should see something similar to Figure 1 below, the 2D Stealth Game from Week 1, in which the player must avoid being detected by NPC guards. This time, we will implement the Guard AI using Behaviour Trees (BT).

. You will need to implement the Guard AI, which will cause them to Patrol, Chase, Investigate and more. **In the starting state, the Guard will not move, and you will get an error message about missing tree nodes – that is as expected.**

If the code does not compile or run, ask staff for assistance.



Figure 1: 2D Stealth Game

Tasks

Simple Sequence Selector

Instead of using a Finite State Machine (FSM) for the Guard AI, this time, we'll use a Behaviour Tree. To begin, you will construct the section of the tree responsible for the guard Patrolling.

In the Week 1 FSM worksheet, the actions created simply called '`Patrol()`' in the `AIActor_Guard` class, and that caused the guard to Patrol continuously between random points. However, that Patrol function was actually performing three tasks.

1. Obtaining a Patrol Point
2. Moving to the Patrol Point
3. Checking when the Patrol Point was reached, and if so, repeating the process

In the Behaviour Tree, we will achieve these steps directly using a Sequence Control Node.

In the starting project, the class `BT_ReasonerNode` is the parent class for all the nodes in the Behaviour Tree. It is a type of Reasoner, which as usual, will select from a list of Options.

The project has three classes that inherit from `BT_ReasonerNode`, each of which will represent one type of control node.

- `BT_SequenceNode`
- `BT_FallbackNode`
- `BT_DecoratorNode`

1

- **Locate and Inspect the `BT_ReasonerNode`, `BT_SequenceNode`, `BT_FallbackNode` and `BT_DecoratorNode` classes**

Each of the Control Node classes has two functions – **Think()** and **Act()**. This is where the custom functionality of each node is implemented.

The **Think()** function will check to see if the Node's conditions are met, while **Act()** will manage the running of the child nodes, and reporting the current status to the parent node.

The **BT_FallbackNode** and **BT_DecoratorNode** classes are complete, but the **Act()** function of the **BT_SequenceNode** needs to be completed.

The **BT_SequenceNode::Act()** function should:

- Get Current Child Node Status
- IF Current Child Node = SUCCESS:
 - IF Current Child = Last Child:
 - Status = SUCCESS
 - ELSE:
 - Move to next Child Node
- ELSE IF Current Child Node = FAILURE
 - Status = FAILURE
- ELSE IF Current Child Node = RUNNING
 - Status = RUNNING
-
- Return Status

2

• **Add code to **BT_SequenceNode::Act()** to match the Pseudocode above**

- Review the Lecture Slides for more details
- Use **BT_FallbackNode::Act()** as a guide, but some adjustments will be needed
 - i. Copying the Debug output from the **FallbackNode** will be useful

As usual, the Actions the Guards can take are defined in the AI Constructor.

In the starting project, there are already two Actions defined in **AIConstructor_BT::DefineActions()**.

- **ActionGetPatrolPath**
- **ActionPatrol**

Two corresponding Options (LeafNodes) are also already created in `AIConstructor_BT::DefineOptions()`:

- `OptionGetPatrolPath`
- `OptionPatrol`

The Behaviour Tree is defined in a similar way to the Decision Trees from Week 2, using the `DefineOptions()` function. See Week 2&3 lecture slides for more information.

The Sequence Node for the Patrol branch is already created:

- `OptionPatrolSeq`

To make the guards patrol, you will need to create the connections between these three options and the Root. There are three connections you need to define.

- `Root -> Option Patrol Seq`
- `Option Patrol Seq -> Option Get Patrol Path`
- `Option Patrol Seq -> Option Patrol`

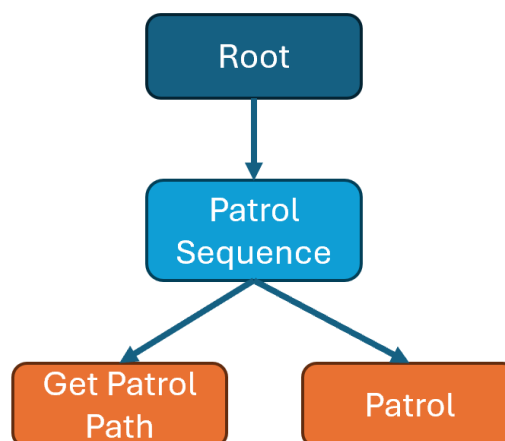


Figure 2: Patrolling

3

- In `DefineOptions()`, use the `AddOptionToTreeNode()` function to create the connections:
 - **Between the Root and the Sequence Selector**
 - **Between the Sequence Selector and the 2 Patrol Options**
 - The first parameter is the Node name (e.g. "Root")
 - The second is the Option Name (e.g. "OptionPatrolSeq")
 - Be sure to use the correct names of the options.

4

- **Run the program.**

- The Guards should now patrol by moving between random points on the map.

If this does not happen, recheck your connections and debug the code.

Decorators

As usual, each Guard has a Blackboard with some data

Key	Type	Example Value	Notes
Energy	float	20	A value in the 0-20 range
CanSeePlayer	Int	0	1 if can see player, otherwise 0
CanHearPlayer	Int	0	1 if can hear player, otherwise 0
IsHealing	Int	0	1 if guard is resting and healing, otherwise 0

When the Guard runs out of Energy, they should Rest so that they recover energy. We can add a new branch to the Behaviour Tree for this.

5

- **Create a new Sequence Control Node**

- This can be created with the function `AddControlNodeByName()`.
 - The first parameter is the name of the new Node/Option
 - The second parameter is the type, which in this case will be a `AIReasonerBase::Sequence`

Next, we will need an Action and Option for the Resting behaviour.

6

- **Create a new Action and Option for Resting**

- The Action can be added to the `DefineActions()` function.
 - The Action should call `Rest()`.
- As the Option is a Leaf Node, it should be created with `AddOptionByName()` in `DefineOptions()`

Notice that the Actions return an **ActionStatus**. This is important, as the Behaviour Tree needs to know which Nodes are Running, have Failed, or have Succeeded.

Decorator Nodes can be used to only allow certain sequences to be executed if their conditions are met. For example, we only want the Resting sequence to take place if the Guard needs to rest.

7

- **Create a new Decorator Control Node**
 - This can be created with the function `AddControlNodeByName()`.
 - i. The first parameter is the name of the Node (Option)
 - ii. The second parameter is the type, which in this case will be a `AIReasonerBase::Decorator`

A Decorator Node requires that some condition is true or false. If it is true, the Sequence will continue, if false, the Node will return Failure. We can use the AI Framework's Considerations to achieve this.

Think in which conditions you want the 'Rest' action to be called.

8

- **Create a condition to this effect in `AIConstructor_BT::DefineConsideration()`.**
 - For a reminder on Considerations, refer to the lecture slides or Week 2 lab sheet.

9

- **Add your Consideration to the Decorator using `AddDecoratorConsideration ()`**
 - The first parameter is the Decorator Name
 - The second parameter is the Consideration Name

New Sequence Connections

You can now create the connections between the Root and your new Sequence Node, and the new Sequence Node the new Decorator and Resting Option using `AddOptionToTreeNode()`.

One important note – the order of the nodes is important as Behaviour Trees operate on a Top to Bottom, Left to Right priority. Given this, consider if you want to add the new Resting Sequence before or after the Patrolling Sequence Node.

10

- **Sketch out your tree to help visualise the Root, Selectors, Decorators and Action/Leaf nodes**

11

- **Create the connection between the Root and your new Sequence Node**
- **Create the connection between the Sequence Node and your new Decorator Node and the Resting Option**

12

- **Run the program.**
 - The Guards should now patrol by moving between random points on the map.
 - When their Energy reaches 0, they should rest (but see 13 below)
 - When their Energy is full, they should resume their patrol

Make sure you understand how the tree is operating, and how the Root Fallback Selector decides which branch to select. The console has status logs to assist with debugging.

13

- **You may notice that the Guard does not immediately rest when their Energy reaches 0.**
 - This is expected for now, but consider why this is.

Leaf Node Conditions

We may wish to make an Action/Leaf Node fail when some condition(s) are not true. For example, if the Guard is carrying out the Patrol task, it should fail if the Energy Level reaches 0.

We can add considerations to leaf node options using the `AddOptionConsideration()` function.

14

- **Create a new Consideration that should be True for the Patrol action to be performed**
- **Add this to the Patrol Option using the `AddOptionConsideration()` function**
 - The first parameter is the Option (Leaf Node) Name
 - The second parameter is the Consideration Name

The guard should now rest immediately upon running out of energy. If this does not happen, debug the tree and code.

Chasing the Player

It will be possible to build the remaining Guard behaviour using more of the same components – Sequence Nodes, Fallback Nodes, Decorator Nodes, Leaf Actions and Considerations.

The complete blackboard data is below. When getting/editing data, make sure to use the correct data type.

Key	Type	Example Value	Notes
Energy	float	20	A value in the 0-20 range
CanSeePlayer	Int	0	1 if can see player, otherwise 0
CanHearPlayer	Int	0	1 if can hear player, otherwise 0
IsHealing	Int	0	1 if guard is resting and healing, otherwise 0
IsRaisingAlarm	Int	0	1 if guard is on the way to the alarm, otherwise 0
HasRaisedAlarm	int	0	1 if guard has raised the alarm, otherwise 0
PlayerHeardAt	Vector2	3,5	The X, Y position the player was last heard
PlayerSeenAt	Vector2	6,2	The X, Y position the player is located

15

- **Expand the tree by adding a branch for Chasing the player**
 - It will be beneficial to make use of Pen and Paper or any software to sketch out the current and expanded Behaviour Tree
 - Think of the Actions, Nodes and Considerations you will need
 - The Chase() function will cause the Guard to chase the player, like in the Week 1 FMS example
 - Make use of the Blackboard data for any conditions

16

- **Run the program.**
 - The Guards should now patrol by moving between random points on the map.
 - When their Energy reaches 0, they should rest
 - When their Energy is full, they should resume their patrol
 - If the Guard sees a player, they should Chase them

If the guards are not yet very effective, think about why.

Like the Patrolling and Energy conditions, you may need to make use of Considerations to stop the Guard performing one task and instead focus on a higher priority one, such as chasing the player.

Completing the Behaviour Tree

The complete list of Actions the Guard can take is:

- GetPatrolPath() & Patrol()
- Chase()
- Rest()
- GetInvestigatePath() & Investigate()
- GetAlarmPath() & RaiseAlarm()
- Sprint()

The typical desired behaviour is:

- The Guards should patrol by moving between random points on the map
- If their Energy reaches 0, they must rest
 - i. If their Energy is full, they should resume their patrol
- If the Guard sees a player, they should Chase them

- The guard should Investigate when it hears a player
 - i. `GetInvestigatePath()` will set the destination to the place the Guard last heard the player
 - ii. `Investigate()` will move to the point last heard
- The Guard has the option of Raising an Alarm when they see a player
 - i. `GetAlarmPath()` will set the destination to the alarm point
 - ii. `RaiseAlarm()` will move the Guard to the Alarm
 - iii. Raising the alarm will cause 2 additional Guards to spawn near the green zone, but may distract the Guard from catching the player
- The guard can `Sprint()` when trying to catch the player
 - i. They will move faster, but use more energy

17

- **Design a Behaviour Tree that incorporate all these Actions**
- **You are free to create the Guard behaviours of your choice**
 - For example, what should happen if the Guard loses sight of the player when chasing them?
 - Should the Guard always recover full health before any other action?
 - You can add more data to the Blackboard as required for status flags or condition checking

Function	Usage
<code>AddOptionByName()</code>	Adding Leaf Nodes
<code>AddControlNodeByName()</code>	Adding Control Nodes
<code>AddOptionToTreeNode()</code>	Creating Connections
<code>AddOptionConsideration()</code>	Adding Conditions to all nodes except Decorator
<code>AddDecoratorConsideration()</code>	Adding Conditions to Decorator Nodes

Table 1: Tree Construction Functions

Bonus Content

Additional Node Types

In the Lectures, we covered two additional node types – Random Selectors and Concurrent/Parallel Selectors.

Template classes are created under the 'AdditionalNodes' filter.

- BT_RandomNode
- BT_ConcurrentNode

- **Implement one or both of these additional control node types.**
 - Use BT_DecoratorNode, BT_SequenceNode and BT_FallbackNode as a guide
 - You will need to update the Think() and Act() functions.
 - Alter your Behaviour Tree to make use of the new nodes

Behaviour Trees vs Decision Trees

In Week 2, we explored Decision Trees.

- **Create a Decision Tree that matches the behaviour from your Behaviour Tree design at Step 17**

- **Compare and contrast Behaviour Trees and Decision Trees:**
 - What was easier to design?
 - Which was easier to adapt?
 - Which allowed for more complex behaviour?

Unreal Engine

Unreal Engine contains its own [Behaviour Tree framework](#).

Each framework can have slight variations in how they use control nodes, conditions and decorators.

- **Research Unreal Engine's Behaviour Tree Framework and note any terminology or implementation differences from the Behaviour Trees as outlined in Chapter 5.4 of "AI for Games".**
 - See [Reading List](#) for details